

0.1 Algorytmy zachłanne

Data: s, f

Result: A – wybrane zajęcia

```

1  $n = \text{len}(s);$ 
2  $A = \{1\};$ 
3  $j = 1;$ 
4 for  $i = 2$  to  $n$  do
5   if  $s_i \geq f_j$  then
6      $A = A \cup \{i\};$ 
7      $j = i;$ 
8   end
9 end
10 return  $A$ 
```

Algorithm 1: Zachłanny wyboru zajęć

Data: C – tablica liczby wystąpień

Result: drzewo prefiksowe

```

1  $n = |C|;$ 
2  $Q = C;$ 
3 for  $i = 1$  to  $n - 1$  do
4    $z = \text{AllocateNode}();$ 
5    $x = \text{left}[z] = \text{ExtractMin}(Q);$ 
6    $y = \text{right}[z] = \text{ExtractMin}(Q);$ 
7    $f[z] = f[x] + f[y];$ 
8    $\text{Insert}(Q, z);$ 
9 end
10 return  $\text{ExtractMin}(Q)$ 
```

Algorithm 2: Huffmana

Data: M – matroid, w – funkcja wag

Result: A – wybrane zajęcia

```

1  $A = \emptyset$ ;
2 uporządkuj  $S[M]$  nierosnąco według wagi  $w$ ;
3 foreach  $x \in S[M]$ , brane w porządku nierosnacym według
   wagi  $w(x)$  do
4   if  $A \cup \{x\} \in \mathcal{F}[M]$  then
5      $A = A \cup \{x\}$ ;
6   end
7 end
8 return  $A$ 

```

Algorithm 3: Szeregowania zadań

Lemat 1. Dla dowolnego zbioru zadań A równoważne są następujące stwierdzenia

1. zbiór A jest niezależny
2. dla $t = 1, 2, \dots, n$ zachodzi nierówność $N_t(A) \leq t$
3. jeśli w A zadania uporządkowane są niemalejąco według dopuszczalnego terminu wykonania, to żadne zadanie nie jest spóźnione.

Twierdzenie 1. Jeśli S jest zbiorem zadań o jednostkowych czasach wykonania, a $\mathcal{C}$ rodziną podzbiorów niezależnych zbioru S , to $(S, \mathcal{C})$ jest matroidem.

Dowód. Każdy podzbiór niezależnego zbioru zadań jest też oczywiście niezależny. Aby wykazać własność wymiany, założmy że A i B są zbiorami niezależnymi oraz że $|B| > |A|$. Niech k będzie największym t takim że zachodzi $N_t(B) > N_t(A)$. Wiadomo, że $N_n(B) = |B|$, $N_n(A) = |A|$ i $|B| > |A|$, więc $k < n$ oraz dla każdego j z przedziału $k+1 \leq j < n$ zachodzi $N_j(B) > N_j(A)$. Stąd B zawiera więcej zadań niż A o dopuszczalnym terminie wykonania

nie większym od $k + 1$. Niech x będzie zadaniem w $B - A$ o dopuszczalnym terminie nie większym od $k + 1$. Niech $A' = A \cup \{x\}$.

Korzystając z własności z lematu 1, wykażemy teraz, że zbiór A' musi być niezależny. Dla $1 \leq i \leq k$ mamy $N_t(A') \leq N_t(A) \leq t$, ponieważ A jest zbiorem niezależnym. Dla $1 \leq i \leq k$ mamy $N_t(A') \leq N_t(B) \leq t$, ponieważ B jest niezależny. Stąd A' jest nie zależny, co kończy dowód że $(S, \mathcal{C})$ jest matroidem. $\square$

0.2 Programowanie dynamiczne

Data: A, B – macierze

Result: $C = A \times B$

```

1 if columns[A]  $\neq$  rows[B] then
2   | return niezgodne rozmiary
3 end
4 for i = 1 to rows[A] do
5   | for j = 1 to columns[B] do
6     |   C[i, j] = 0;
7     |   for k = 1 to columns[A] do
8       |     | C[i, j] = C[i, j] + A[i, k]B[k, j];
9       |   end
10    | end
11 end
12 return C
```

Algorithm 4: Mnożenia macierzy

Data: p – ciąg macierzy

Result: m – tablica kosztów, s – tablica indeksów

```

1  $n = \text{length}[p] - 1;$ 
2 for  $i = 1$  to  $n$  do
3    $m[i, i] = 0;$ 
4 end
5 for  $l = 2$  to  $n$  do
6   for  $i = 1$  to  $n - l + 1$  do
7      $j = i + l = 1;$ 
8      $m[i, j] = \infty;$ 
9     for  $k = i$  to  $j - 1$  do
10       $q = m[i, k] + m[k + 1, j] + p_{i-1}p_kp_j;$ 
11      if  $a < m[i, j]$  then
12         $m[i, k] = q;$ 
13         $s[i, j] = k;$ 
14      end
15    end
16  end
17 end
18 return  $m, s$ 

```

Algorithm 5: MatrixChainOrder

$$p = (30, 35, 15, 5, 10, 20, 25)$$

$$m(1, 1) = m(2, 2) = m(3, 3) = m(4, 4) = m(5, 5) = 0$$

$$m(1, 2) = m(1, 1) + m(1, 2) + p_0p_1p_2 = 0 + 0 + 30 \cdot 35 \cdot 15 = 15750$$

$$m(2, 3) = 35 \cdot 15 \cdot 5 = 2625$$

$$m(3, 4) = 15 \cdot 5 \cdot 10 = 750$$

$$m(4, 5) = 5 \cdot 10 \cdot 20 = 1000$$

$$m(5, 6) = 10 \cdot 20 \cdot 25 = 5000$$

$$m(1, 3) = \min \left(\begin{array}{lcl} m(1, \mathbf{1}) + m(2, 3) + p_0 p_1 p_3 & & \\ = 0 + 2625 + 30 \cdot 25 \cdot 5 & = & \mathbf{7875} \\ m(1, 2) + m(3, 3) + p_0 p_2 p_3 & & \\ = 15750 + 0 + 30 \cdot 15 \cdot 5 & = & 18000 \end{array} \right) = 7875$$

$$m(2, 4) = \min \left(\begin{array}{lcl} m(2, 2) + m(3, 4) + p_1 p_2 p_4 & & \\ = 0 + 750 + 35 \cdot 15 \cdot 10 & = & 6000 \\ m(2, \mathbf{3}) + m(4, 4) + p_1 p_3 p_4 & & \\ = 2625 + 0 + 35 \cdot 5 \cdot 10 & = & \mathbf{4375} \end{array} \right) = 4375$$

$$m(3, 5) = \min \left(\begin{array}{lcl} m(3, \mathbf{3}) + m(4, 5) + p_2 p_3 p_5 & & \\ = 0 + 1000 + 15 \cdot 5 \cdot 20 = \mathbf{2500} & & \\ m(3, 4) + m(5, 5) + p_2 p_4 p_5 & & \\ = 750 + 0 + 15 \cdot 10 \cdot 20 = 3750 & & \end{array} \right) = 2500$$

$$m(4, 6) = \min \left(\begin{array}{lcl} m(4, 4) + m(5, 6) + p_3 p_4 p_6 & & \\ = 0 + 5000 + 5 \cdot 10 \cdot 25 & = & 6250 \\ m(4, \mathbf{5}) + m(6, 6) + p_3 p_5 p_6 & & \\ = 1000 + 0 + 5 \cdot 20 \cdot 25 & = & \mathbf{3500} \end{array} \right) = 3500$$

$$m(1, 4) = \min \left(\begin{array}{lcl} m(1, 1) + m(2, 4) + p_0 p_1 p_4 & & \\ = 0 + 4375 + 30 \cdot 35 \cdot 10 & = & 14875 \\ m(1, 2) + m(3, 4) + p_0 p_2 p_4 & & \\ = 15750 + 750 + 30 \cdot 15 \cdot 10 & = & 21000 \\ m(1, \mathbf{3}) + m(4, 4) + p_0 p_3 p_4 & & \\ = 7875 + 0 + 30 \cdot 5 \cdot 10 & = & \mathbf{9375} \end{array} \right) = 9375$$

$$m(2, 5) = \min \left(\begin{array}{lcl} m(2, 2) + m(3, 5) + p_0 p_1 p_4 & & \\ = 0 + 2500 + 35 \cdot 15 \cdot 20 & = & 13000 \\ m(2, \mathbf{3}) + m(4, 5) + p_0 p_2 p_4 & & \\ = 2625 + 1000 + 35 \cdot 5 \cdot 20 & = & \mathbf{7125} \\ m(2, 4) + m(5, 5) + p_0 p_3 p_4 & & \\ = 4375 + 0 + 35 \cdot 5 \cdot 20 & = & 7875 \end{array} \right) = 7125$$

$$m(3, 6) = \min \left(\begin{array}{lcl} m(3, \mathbf{3}) + m(4, 6) + p_2 p_3 p_6 & & \\ = 0 + 3500 + 15 \cdot 5 \cdot 25 & = & \mathbf{5375} \\ m(3, 4) + m(5, 6) + p_2 p_4 p_6 & & \\ = 750 + 5000 + 15 \cdot 10 \cdot 25 & = & 9500 \\ m(3, 5) + m(6, 6) + p_2 p_5 p_6 & & \\ = 2500 + 0 + 15 \cdot 20 \cdot 25 & = & 1000 \end{array} \right) = 5375$$

$$m(1, 5) = \min \left(\begin{array}{lcl} m(1, 1) + m(2, 5) + p_0 p_1 p_5 & & \\ = 0 + 7125 + 30 \cdot 35 \cdot 20 & = & 28125 \\ m(1, 2) + m(3, 5) + p_0 p_2 p_5 & & \\ = 15750 + 2500 + 30 \cdot 15 \cdot 20 & = & 27250 \\ m(1, \mathbf{3}) + m(4, 5) + p_0 p_3 p_5 & & \\ = 7875 + 1000 + 30 \cdot 5 \cdot 20 & = & \mathbf{11875} \\ m(1, 4) + m(5, 5) + p_0 p_4 p_5 & & \\ = 9375 + 0 + 30 \cdot 10 \cdot 20 & = & 15375 \end{array} \right) = 11875$$

$$m(2, 6) = \min \left(\begin{array}{lcl} m(2, 2) + m(3, 6) + p_1 p_2 p_6 & & \\ = 0 + 5375 + 35 \cdot 15 \cdot 25 & = & 18500 \\ m(2, \mathbf{3}) + m(4, 6) + p_1 p_3 p_6 & & \\ = 2625 + 3500 + 35 \cdot 5 \cdot 25 & = & \mathbf{10500} \\ m(2, 4) + m(5, 6) + p_1 p_4 p_6 & & \\ = 4375 + 5000 + 35 \cdot 10 \cdot 25 & = & 18125 \\ m(2, 5) + m(6, 6) + p_1 p_5 p_6 & & \\ = 7125 + 0 + 35 \cdot 20 \cdot 25 & = & 24625 \end{array} \right) = 10500$$

$$m(1, 6) = \min \left(\begin{array}{lcl} m(1, 1) + m(2, 6) + p_0 p_1 p_6 & & \\ = 0 + 10500 + 30 \cdot 35 \cdot 25 & = & 36750 \\ m(1, 2) + m(3, 6) + p_0 p_2 p_6 & & \\ = 15750 + 5375 + 30 \cdot 15 \cdot 25 & = & 32375 \\ m(1, \mathbf{3}) + m(4, 6) + p_0 p_3 p_6 & & \\ = 7875 + 3500 + 30 \cdot 5 \cdot 25 & = & \mathbf{15125} \\ m(1, 4) + m(5, 6) + p_0 p_4 p_6 & & \\ = 9375 + 5000 + 30 \cdot 10 \cdot 25 & = & 21875 \\ m(1, 5) + m(6, 6) + p_0 p_5 p_6 & & \\ = 11875 + 0 + 30 \cdot 20 \cdot 25 & = & 26875 \end{array} \right) = 15125$$

$((A_1(A_2A_3))((A_4A_5)A_6))$

Data: A, s, i, j

```

1 if  $j < i$  then
2    $X = \text{MatrixChainMultiply}(A, s, i, s[i, j]);$ 
3    $Y = \text{MatrixChainMultiply}(A, s, s[i, j] + 1, j);$ 
4   return  $X \times Y$ 
5 else
6   return  $A_i$ 
7 end
```

Algorithm 6: MatrixChainMultiply

Data: A – ciąg

Result: s – suma podciagu

```

1  $n = \text{length}[A];$ 
2  $s = 0;$ 
3  $w = 0;$ 
4 for  $i = 1$  to  $n$  do
5   if  $w > 0$  then
6      $w = w + a[i]$ 
7   else
8      $w = a[i]$ 
9   end
10  if  $w > s$  then
11     $s = w$ 
12  end
13 end
14 return  $s$ 

```

Algorithm 7: Podciąg spójny o największej sumie

Data: $A = (-5, 3, -2, 3, -1)$

```

1  $n = 5;$ 
2  $s = 0, w = 0;$ 
3  $i = 1, w = 0; s = 0;$ 
4  $i = 2, w = 3; s = 3;$ 
5  $i = 3, w = 1; s = 3;$ 
6  $i = 4, w = 4; s = 4;$ 
7  $i = 5, w = 3; s = 4;$ 

```

Result: 4

0.3 Dziel i zwyciężaj

- 1 Dla zadanego zbioru punktów Q , $|Q| = n > 2$. Szukamy pary punktów $p_1 = (x_1, y_1)$, $p_2 = (x_2, y_2)$ których odległość jest najmniejsza.
- 2 Przy każdym rekurencyjnym wywołaniu algorytmu dane wejściowe stanowi podzbiór $P \subset Q$ oraz tablice X i Y , z których każda zawiera wszystkie punkty z wejściowego podzbioru P . Punkty w tablicy X są uporządkowane rosnąco względem współrzędnej x , a w tablicy Y – względem współrzędnej y . W konkretnym wywołaniu rekurencyjnym dla danych P , X i Y sprawdzamy najpierw, czy $|P| \leq 3$. Jeśli tak to sprawdzamy wszystkie punkty metodą "siłową" i zwracamy najmniejszy wyniki. W przeciwnym przypadku realizujemy zasadę "dziel i zwyciężaj" w następujący sposób.

Dziel Znajdujemy prostą l , dzielącą zbiór punktów P na dwa zbiory P_L i P_R takie, że $P_L = \lceil \frac{P}{2} \rceil$, $P_R = \lfloor \frac{P}{2} \rfloor$. Wszystkie punkty ze zbioru P_L leżą na lewo od prostej l , a wszystkie punkty ze zbioru P_R na prawo od prostej l . Tablica X zostaje podzielona na tablice X_L i X_R , zawierające odpowiednio punkty P_L i P_R posortowane rosnąco względem współrzędnej x . Podobnie dzielimy tablicę Y na tablice Y_L i Y_R , składające się z odpowiednio z P_L i P_R posortowanych względem współrzędnej y .

Zwyciężaj Po podzieleniu P na P_L i P_R wykonujemy dwa wywołania rekurencyjne – jedno w celu znalezienia pary najmniej odległych punktów w P_L , a drugie w P_R . Oznaczamy odległości par najbliższych sobie punktów obliczone dla P_L i P_R odpowiednio przez δ_L i δ_R . i niech $\delta = \min(\delta_L, \delta_R)$

Połącz Para najmniej odległych punktów jest albo parą o odległości δ znaną przez jedno z wywołań rekurencyjnych, albo parą, w której jeden punkt należy do P_L , a drugi do P_R . Algorytm sprawdza czy istnieje taka para o odległości mniejszej niż δ . Zauważmy że jeśli istnieje para punktów o odległości miniejszej niż δ , to obydwa punkty nie mogą być dalej niż δ od prostej l . Jak zatem widać obydwa punkty muszą się mieścić wewnątrz pionowego pasa szerokości 2δ wokół prostej l . W celu znalezienia takiej pary, o ile istnieje, algorytm wykona następujące operacje:

Twierdzenie 2. *Poprawność algorytmu jest oczywista, z wyjątkiem dwóch kwestii. Po pierwsze, kończąc rekursję kiedy $|P| \leq 3$, zapewniamy sobie to, że nigdy nie będziemy dzielić zbioru składającego się tylko z jednego punktu. Drugą kwestią jest to, że musimy sprawdzać tylko 7 punktów następujących po każdym punkcie p w tablicy Y' ; teraz zajmiemy się dowodem tej własności.*

Przypuśćmy, że na pewnym poziomie rekursji parę najmniej odległych punktów stanowią $p_L \in P_L$ i $p_R \in P_R$. Zatem odległość δ' między p_L a p_R jest ostro mniejsza niż delta. Punkt p_L musi leżeć na prostej l lub na lewo od niej, ale bliżej niż δ . Podobnie, p_R leży na l lub na prawo od niej i bliżej niż δ . Co więcej, odległość w poziomie p_L i p_R jest mniejsza niż δ . Zatem, jak widać p_L i p_R mieszczą się w prostokącie $\delta \times 2\delta$ położonym symetrycznie względem prostej l .¹

Pokażemy teraz, że w owym prostokącie $\delta \times 2\delta$ może znajdować się co najwyżej 8 punktów ze zbioru P . Rozważmy kwadrat $\delta \times \delta$ stanowiący lewą połowę tego prostokąta. Ponieważ wszystkie punkty zbioru P_L są odległe od siebie przynajmniej o δ , wewnątrz tego kwadratu mogą znajdować się co najwyżej cztery punkty. Podobnie, co najwyżej cztery punkty ze zbioru P_R mogą znajdować się w kwadracie $\delta \times \delta$ stanowiącym prawą połowę prostokąta. Zatem w sumie na w prostokącie $\delta \times \delta$ może znajdować się co najwyżej 8 punktów ze zbioru P .²

Wiedząc, że wewnątrz prostokąta może się znajdować co najwyżej 8 punktów ze zbioru P , łatwo stwierdzić, że wystarczy sprawdzać po 7 punktów następujących po każdym punkcie w tablicy Y' . Trzymając się założenia, że para najmniej odległych punktów to p_L i p_R , możemy bez straty ogólności przyjąć, że p_L poprzedza p_R w tablicy

¹W prostokącie tym mogą być również inne punkty.

²Zauważmy, że ponieważ punkty na prostej l mogą należeć zarówno do P_L , jak i P_R na l mogą leżeć nawet 4 punkty. Liczba ta zostaje osiągnięta, jeśli istnieją dwie pary pokrywających się punktów, z których każda składa się z jednego punktu z P_L i jednego z P_R , jedna para znajduje się na przecięciu l z górnym, a druga z dolnym bokiem prostokąta.

Y' . Wówczas, nawet jeśli p_L występuje najwcześniej, a p_R najpóźniej jak można w tablicy Y' , punkt p_R jest na jednej z 7 pozycji następujących po p_L . Wykazaliśmy zatem poprawność algorytmu znajdowania pary najmniej odległych punktów.

0.4 Algorytmy aproksymacyjne

Data: X – zbiór dla którego szukamy pokrycie, $\mathcal{F}$ – rodzina zbiorów;

Result: $\mathcal{C} \in \mathcal{F}$ – pokrycie X

```

1  $U = X$ ;
2  $\mathcal{C} = \emptyset$ ;
3 while  $U \neq \emptyset$  do
4   |   wybierz  $S \in \mathcal{F}$ , które maksymalizuje  $|S \cap U|$ ;
5   |    $U = U - S$ ;
6   |    $\mathcal{C} = \mathcal{C} \cup \{S\}$ ;
7 end
8 return  $\mathcal{C}$ 
```

Algorithm 9: GreedySetCover

Twierdzenie 3. Ograniczenie względne algorytmu wynosi $H(\max\{|S| : S \in \mathcal{C}\})$

Dowód. Dowód przeprowadzimy, związując z każdym zbiorem wybieranym przez algorytm pewien koszt, rozdzielając ten koszt między elementy pokrywane po raz pierwszy i wykorzystując owe koszty do wyprowadzenia zapowiedzianej zależności między rozmiarem optymalnego pokrycia $\mathcal{C}^*$ a rozmiarem pokrycia $\mathcal{C}$ znajdowanego przez algorytm. Niech S_i oznacza i -ty podzbiór wybierany przez procedurę dodanie przez algorytm S_i do $\mathcal{C}$ ma koszt jednostkowy. Koszt wybrania S_i rozdzielamy równo między elementy pokrywane przez S_i po raz pierwszy. Dla każdego $x \in X$ oznaczamy koszt związany z elementem x jako c_x . Każdemu elementowi przypisujemy koszt tylko jeden raz, kiedy zostaje pokryty po raz pierwszy.

Jeśli x jest pokryty po raz pierwszy przez S_i to

$$c_x = \frac{1}{|S_i - (S_1 \cup S_2 \cup \dots \cup S_{i-1})|}$$

Algorytm znajduje pokrycie $\mathcal{C}$ o całkowitym koszcie $|\mathcal{C}|$, a koszt ten zostaje rozdzielony między elementy zbioru X . Koszt optymalnego pokrycia wynosi

$$\sum_{S \in \mathcal{C}^*} \sum_{x \in S} c_x$$

Ponieważ optymalne pokrycie $\mathcal{C}^*$ także pokrywa X , mamy

$$|\mathcal{C}| = \sum_{x \in X} c_x \leq \sum_{S \in \mathcal{C}^*} \sum_{x \in S} c_x \quad (1)$$

Dalsza część dowodu opiera się na następującej istotnej nierówności, której prawdziwość wykażemy poniżej.

$$\forall S \in \mathcal{F} \quad \sum_{x \in S} c_x \leq H(|S|) \quad (2)$$

Z nierówności 1 wynika że

$$|\mathcal{C}| \leq \sum_{S \in \mathcal{C}^*} H(|S|) \leq |\mathcal{C}^*| \cdot H(\max\{|S| : S \in \mathcal{F}\})$$

Co dowodzi tezy twierdzenia. Pozostaje tylko wykazać nierówność 2. Dla dowolnego zbioru $S \in \mathcal{F}$ oraz $i = 1, 2, \dots, |\mathcal{C}|$ niech $u_i = |S - (S_1 \cup S_2 \cup \dots \cup S_i)|$ będzie liczbą elementów zbioru S nadal nie pokrytych po wybraniu przez algorytm zbiorów $S_1, S_2, \dots, S_i$. Zdefiniujmy dodatkowo $u_0 = |S|$ jako liczbę elementów S nie pokrytych na samym początku. Niech k będzie najmniejszym takim indksem, że $u_k = 0$, a więc każdy element S jest pokryty przynajmniej przez jeden spośród zbiorów $S_1, S_2, \dots, S_k$. Wówczas dla

$i = 1, 2, \dots, k$ zachodzi $u_{i-1} \geq u_i$, a zbiór S_i pokrywa po raz pierwszy $u_{i-1} - u_i$ elementów S . Zatem

$$\sum_{x \in S} c_x = \sum_{i=1}^k (u_{i-1} - u_i) \cdot \frac{1}{|S_i - (S_1 \cup S_2 \cup \dots \cup S_{i-1})|}$$

Zauważmy, że

$$|S_i - (S_1 \cup S_2 \cup \dots \cup S_{i-1})| \geq |S - (S_1 \cup S_2 \cup \dots \cup S_{i-1})| = u_{i-1}$$

ponieważ zachłanny wybór S_i zapewnia, że S nie pokrywa większej liczby nowych elementów niż S_i ³. Mamy zatem

$$\sum_{x \in S} c_x \leq \sum_{i=1}^k (u_{i-1} - u_i) \cdot \frac{1}{u_{i-1}}$$

Oszacujmy teraz tę wielkość następująco:

$$\begin{aligned} \sum_{x \in S} c_x &\leq \sum_{i=1}^k (u_{i-1} - u_i) \cdot \frac{1}{u_{i-1}} \\ &= \sum_{i=1}^k \sum_{j=u_i+k}^{u_{i-1}} \frac{1}{u_{i-1}} \\ &\leq \sum_{i=1}^k \sum_{j=u_i+k}^{u_{i-1}} \frac{1}{j} \\ &= \sum_{i=1}^k \left(\sum_{j=1}^{u_{i-1}} \frac{1}{j} - \sum_{j=1}^{u_i} \frac{1}{j} \right) \\ &= \sum_{i=1}^k (H(u_{i-1}) - H(u_i)) \end{aligned}$$

³w przeciwnym razie zamiast S_i zostałyby wybrany S

$$\begin{aligned}
&= H(u_0) - H(u_k) \\
&= H(u_0) - H(0) \\
&= H(u_0) \\
&= H(|S|)
\end{aligned}$$

Co kończy dowód nierówności. □

Data: G – graf

Result: C – znalezione pokrycie wierzchołkowe

```

1  $C = \emptyset$ ;
2  $E' = E[G]$ ;
3 while  $E' \neq \emptyset$  do
4   |   wybierz dowolną krawędź  $(u, v) \in E'$ ;
5   |    $C = C \cup \{u, v\}$ ;
6   |   usuń z  $E'$  wszystkie krawędzie incydentne z  $u$  lub  $v$ 
7 end
8 return  $C$ 

```

Algorithm 10: ApproxVertexCover

Twierdzenie 4. *Ograniczenie względne algorytmu wynosi 2*

Dowód. Zbiór C skonstruowany przez procedurę jest pokryciem wierzchołkowym, ponieważ pętla w algorytmie jest wykonywana tak długo, aż każda krawędź z $E[G]$ zostanie pokryta przez jakiś wierzchołek z C .

Żeby się przekonać że algorytm oblicza pokrycie wierzchołkowe co najwyżej dwukrotnie większe od optymalnego, oznaczmy przez A zbiór krawędzi wybranych w wierszu 4 procedury. Żadne dwie krawędzie z A nie mają wspólnego wierzchołka, ponieważ za każdym razem, kiedy w wierszu 4 jest wybierana krawędź, wszystkie inne krawędzie incydentne z jej końcami zostają usunięte z E' w wierszu 6. Każde wykonanie operacji w wierszu 5 dodaje dwa nowe wierzchołki do C , zatem $|C| = 2|A|$. Aby jednak pokryć wszystkie krawędzie z A , każde pokrycie wierzchołkowe – w szczególności

pokrycie optymalne C^* – musi zawierać przynajmniej jeden z końców każdej krawędzi z A . Ponieważ żadne dwie krawędzie w A nie mają wspólnego końca, żaden wierzchołek pokrycia nie jest incydentny w więcej niż jedną krawędzią z A . Mamy zatem $|A| \leq |C^*|$ i $|C| \leq 2|C^*|$, co kończy dowód twierdzenia. $\square$

Data: S – zbiór, t – szukana suma

Result: największa możliwa $\leq t$

```

1  $n = |S|$ ;
2  $L_0 = (0)$ ;
3 for  $i = 0$  to  $n$  do
4    $L_i = \text{MergeLists}(L_{i-1}, L_{i-1} + x_i)$ ;
5   usuń z  $L_i$  wszystkie elementy większe niż  $t$ 
6 end
7 return  $\max(L_n)$ 
```

Algorithm 11: ExactSubsetSum

Data: L – list, $\delta \in (0, 1)$ – tolerancja

Result: L' – lista pozostałych elementów

```

1  $m = |L|$ ;
2  $L' = \{y_1\}$ ;
3  $last = y_1$ ;
4 for  $i = 2$  to  $m$  do
5   if  $last < (1 - \delta)y_i$  then
6     dołącz  $y_i$  na koniec listy  $L'$ ;
7      $last = y_i$ ;
8   end
9 end
10 return  $L'$ 
```

Algorithm 12: Trim

Data: S – zbiór, t – szukana suma, ϵ – dokładność

Result: suma $\leq t$

```

1  $n = |S|;$ 
2  $L_0 = (0);$ 
3 for  $i = 1$  to  $n$  do
4    $L_i = \text{MergeLists}(L_{i-1}, L_{i-1} + x_i);$ 
5    $L_i = \text{Trim}(L_i, \frac{\epsilon}{n});$ 
6   usuń z  $L_i$  wszystkie elementy większe niż  $t$ 
7 end
8 return  $\max(L_n)$ 
```

Algorithm 13: ApproxSubsetSum

Data: $S = (104, 102, 201, 101), t = 308$

```

1  $L_0 = (0);$ 
2  $L_1 = (0, 104);$ 
3  $L_2 = (0, 102, 104, 206);$ 
4  $L_3 = (0, 102, 104, 201, 206, 303, 305, 407);$ 
5  $L_4 =$ 
    $(0, 101, 102, 104, 201, 203, 205, 206, 302, 303, 305, 307, 404, 406);$ 
```

Result: $\max(L_4) = 307$

Data: $S = (104, 102, 201, 101), t = 308, \epsilon = 0.4$

```

1  $L_0 = (0);$ 
2  $L_1 = (0, 104);$ 
3  $L_2 = \text{Trim}((0, 102, 104, 206), \frac{0.4}{2.4}) = (0, 102, 206);$ 
4  $L_3 = (0, 102, 201, 206, 303, 406);$ 
5  $L_4 = (0, 101, 102, 201, 203, 302, 303, 404);$ 
```

Result: $\max(L_4) = 302$

0.5 Algorytmy grafowe znajdowania skojarzenia

Twierdzenie 5 (Berge’a o ścieżkach powiększających). *Skojarzenie M jest maksymalne gdy nie istnieje względem niego żadna ścieżka powiększająca.*

Dowód. Załóżmy przeciwnie, że istnieje skojarzenie M' liczniejsze niż M . Rozważmy graf $G' = (V, M \oplus M')$. Zauważmy, że w G' każdy wierzchołek ma stopień co najwyżej 2, w związku z tym G' składa się z rozłącznych ścieżek i cykli. Na każdym cyklu występuje tyle samo krawędzi z M co z M' . Natomiast w ścieżka może występować co najwyżej o jedną krawędź więcej z któregoś skojarzenia. W grafie G' jest więcej krawędzi z M' niż z M , a zatem musi też istnieć ścieżka, na której jest więcej krawędzi z M' . Musi to być oczywiście ścieżka powiększająca. $\square$

Data: $G = (V_1 \cup V_2, E)$ – graf dwudzielny;

Result: M – skojarzenie

```

1  $M = \emptyset$ ;
2 repeat
3   | znajdź ścieżkę powiększającą  $p$  dla  $G, M$ ;
4   | if  $p \neq \emptyset$  then
5   |   |  $M = M \oplus p$ ;
6   | end
7 until  $p = \emptyset$ ;
8 return  $M$ 
```

Algorithm 14: Znajdujący maksymalne skojarzenie w grafie dwudzielnym

Data: $G = (V_1 \cup V_2, E)$ – graf dwudzielny; M – skojarzenie G ;

Result: p – ścieżka powiększająca lub $\emptyset$ jeśli nie znaleziono

- 1 $V'_1 =$ zbiór wierzchołków wolnych w V_1 ;
- 2 $V'_2 =$ zbiór wierzchołków wolnych w V_2 ;
- 3 skonstruuuj graf skierowany $G_M = (V_1 \cup V_2, E_M)$;
- 4 znajdź ścieżkę p z V'_1 do V'_2 w G_M ;
- 5 **if** p *nie istnieje* **then**
- 6 **return** $\emptyset$
- 7 **end**
- 8 usuń cykle z p tak aby p była ścieżką prostą;
- 9 **return** p

Algorithm 15: Znajdujący ścieżkę powiększającą w grafu dwudzielnym

Data: $G = (V, E)$ – graf;
Result: M – skojarzenie

```

1  $M = \emptyset$ ;
2 las  $L$  to zbiór wierzchołków wolnych;
3 Loop
4   if istnieje zewnętrzny wierzchołek  $x \in L$  sąsiedni z
      wierzchołkiem  $y \notin L$  then
5     |   znajdź wierzchołek  $z$  taki, że  $yz \in M$ ;
6     |    $L = L + xy + yz$ ;
7   else
8     |   if  $x_1, x_2 \in L$  to wierzchołki zewnętrzne połączone
      krawędzią then
9       |   if  $x_1, x_2$  należą do różnych składowych  $L$  then
10      |   |   niech  $P_i$  to ścieżka z  $x_i$  do korzenia jego
11      |   |   składowej;
12      |   |   rozwiń wszystkie ściągnięte cykle w grafie  $G$ ;
13      |   |    $M = M \oplus (P_1 + P_2 + x_1x_2)$ ;
14      |   |   las  $L$  to zbiór wierzchołków wolnych;
15      |   else
16      |   |   niech  $C$  będzie cyklem utworzonym przez
17      |   |   krawędź  $x_1x_2$  oraz ścieżkę  $x - y$  w  $L$ ;
18      |   |   niech  $P$  będzie ścieżką w  $L$  łączącą  $C$  z
19      |   |   korzeniem drzewa;
20      |   |    $M = M \oplus P$ ;
21      |   |   utwórz graf  $G'$  poprzez ściągnięcie  $C$ ;
22      |   |    $G = G'$ ;
23      |   end
24     |   else
25     |   |   rozwiń wszystkie ściągnięte cykle w grafie  $G$ ;
26     |   |   return  $M$ 
27     |   end
28   end

```

Algorithm 16: Edmondsa